Step 4 — An Analysis Agent That Turns Raw Research Into Organized Findings — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: raw research notes are a flat list of facts; a good article needs structure. The Analysis agent groups the notes into themes with a model call — **without losing any fact**, including ones that don't cleanly fit a theme. This is the first step that feeds one already-built agent's *real* output (Research's notes) into another, so an interface mismatch between the two shows up now, in isolation, before a Supervisor graph hides it inside a bigger run. **If not:** the Writer would get an unordered list, or silently lose a fact that didn't fit a theme.

After this step the project runs: research → analysis → write → review. Files not shown here don't change.

Solution — themes, every fact kept, checked on Research's real output

Story — `prompts/analysis.txt`: this spells out the rules a schema alone can't: group into 2-4 themes, keep **every** fact and its `[source]` tag somewhere, and use a catch-all "Other" theme for facts that genuinely don't fit elsewhere — so nothing is ever silently dropped. **If not:** the article would quietly lose facts between Research and the Writer.

`src/prompts/analysis.txt`

```
You are the Analyst on a content team. Organize the research notes below
into 2 to 4 themes. Under each theme, list its facts as bullet points.
Keep EVERY fact and its [source] tag - do not drop a fact just because it
does not fit a theme neatly; put it under "Other" instead.

Research notes:
{notes}
```

Story — `agents/analysis_agent.py`: this is a plain LCEL chain with no tool attached, because organizing existing notes into themes is reasoning over text already in hand, not looking something up. **If not:** you'd give it a tool it never needs, adding complexity with no benefit.

```
# src/agents/analysis_agent.py
# The Analysis agent. run_analysis(notes) is a plain LCEL chain that groups
# the research notes into themes while keeping every fact.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.models.llm import get_model
from src.utils.prompts import load_prompt

def run_analysis(notes):
    prompt = ChatPromptTemplate.from_template(load_prompt("analysis"))
    chain = prompt | get_model() | StrOutputParser()
    return chain.invoke({"notes": notes})
```

Changed from Step 3: `main.py` runs Analysis between Research and the writing loop.

```
# main.py
# The file you run: python main.py
# Step 4 version: topic -> research -> analysis -> write and review.
import sys

from src.agents.analysis_agent import run_analysis
from src.agents.research_agent import run_research
from src.agents.reviewer_agent import PASS_SCORE, run_reviewer
from src.agents.writer_agent import run_writer
from src.config import load_config
from src.exceptions import MissingConfigError
from src.utils.logger import get_logger

def write_with_review(findings):
    logger = get_logger(__name__)
    result = {
        "draft": "",
        "accepted": False,
        "feedback_history": [],
        "revision_count": 0,
    }
    feedback = ""
    for round_number in range(1, 3 + 1):
        result["draft"] = run_writer(findings, feedback)
        verdict = run_reviewer(findings, result["draft"])
        logger.info(f"round {round_number}: score {verdict.score}")
        if verdict.score >= PASS_SCORE:
            result["accepted"] = True
            return result
        feedback = verdict.feedback
```

```
        result["feedback_history"].append(feedback)
        result["revision_count"] = result["revision_count"] + 1
    return result

def print_report(result):
    if result["accepted"]:
        print("\nFinal article:\n")
        print(result["draft"])
    else:
        attempts = result["revision_count"]
        print(f"\nCouldn't agree after {attempts} revisions. Feedback:")
        number = 1
        for feedback in result["feedback_history"]:
            print(f"  round {number}: {feedback}")
            number = number + 1
        print("\nLast draft (NOT approved):\n")
        print(result["draft"])

def main():
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    topic = input("Topic: ").strip()
    notes = run_research(topic)
    if notes == "":
        print("No article: nothing is known about this topic yet.")
        return
    # new in Step 4: Analysis organizes the notes before the Writer
    findings = run_analysis(notes)
    print(f"Findings:\n{findings}\n")
    result = write_with_review(findings)
    print_report(result)

if __name__ == "__main__":
    main()
```

Changed from Step 3: the integration test adds case 5 — Research's **real** output goes in, and the `[source]` tags are counted before and after. Same count = no fact dropped.

```
# tests/integration/test_live_agents.py
# Runs the agents against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
```

```
from src.agents.analysis_agent import run_analysis
from src.agents.research_agent import run_research
from src.agents.reviewer_agent import PASS_SCORE, run_reviewer
from src.agents.writer_agent import run_writer
from src.config import load_config

FINDINGS = (
    "Range:\n- A battery gives 40 to 100 km per charge. [battery-review]\n"
    "Rules:\n- EU e-bike motors are limited to 25 km/h. [eu-rules]\n"
)

def test_writer_keeps_sources():
    draft = run_writer(FINDINGS)
    result = "OK"
    if "[" not in draft:
        result = "WRONG"
    print(f"case 1 (writer keeps sources) -> {result}")

def test_reviewer_rejects_weak_draft():
    verdict = run_reviewer(FINDINGS, "E-bikes are good.")
    result = "OK"
    if verdict.score >= PASS_SCORE:
        result = "WRONG"
    print(f"case 2 (weak draft fails)     -> {result}")

def test_research_keeps_sources():
    notes = run_research("electric bikes")
    result = "OK"
    if notes.count("[") < 5:
        result = "WRONG"
    print(f"case 3 (known topic sourced)  -> {result}")

def test_research_unknown_topic():
    notes = run_research("deep sea mining")
    result = "OK"
    if notes != "":
        result = "WRONG"
    print(f"case 4 (unknown topic empty)  -> {result}")

def test_analysis_keeps_every_fact():
    notes = run_research("electric bikes")
    findings = run_analysis(notes)
    result = "OK"
    if findings.count("[") != notes.count("["):
        result = "WRONG"
    print(f"case 5 (analysis keeps facts) -> {result}")
```

```
if __name__ == "__main__":
    load_config()
    test_writer_keeps_sources()
    test_reviewer_rejects_weak_draft()
    test_research_keeps_sources()
    test_research_unknown_topic()
    test_analysis_keeps_every_fact()
```

Run it (from the repo root):

```
python -m tests.integration.test_live_agents
python main.py
```

Expected app flow: `Topic:` → research notes → themed findings → a reviewed article (or "nothing is known" for an unknown topic).